Helix OTA Server — §11.4.169 Test-Type Coverage Audit

Revision: 1 **Last modified:** 2026-07-09T00:00:00Z **Scope:** `server/` (Go control plane module `github.com/HelixDevelopment/helix_ota/server`) ONLY. This is a READ-ONLY audit — no test was added, no source was modified, no git command was run. All findings below are either (a) read directly from test source, or (b) captured from real command execution on this host on 2026-07-09/10. **Method:** enumerate every `*_test.go` under `server/`, read what each exercises, run the real suite (`go build`, `go test ./...`, `go test ./... -race`, `go vet`, `gofmt -l`, `go test -bench`), classify against the §11.4.169 closed test-type set, and report gaps with concrete descriptions. No coverage-% claim is made without a captured run backing it (§11.4.6). **Companion doc:** `docs/research/test-coverage-audit-20260623/REPORT.md` (2026-06-23) covers **code coverage %** + mutation-proof from a prior round; this document covers the §11.4.169 **test-TYPE** closed set specifically and is a different cut of the same codebase. Cross-referenced, not duplicated.

---

1. Existing test inventory (file → functions → test-type classification)

`find server -name '*_test.go'` returned **73 files**, `grep -c '^func (Test|Benchmark|Fuzz)'` returned **489 functions**: **482 `Test*`**, **7 `Benchmark*`**, **0 `Fuzz*`**.

1.1 `cmd/` — binary smoke tests (unit + light e2e)

File	Representative functions	Type
`cmd/applyport/main_test.go`	`TestRun_UnknownCommand`, `TestBuildVerifier`, `TestSmoke_NoArgsUsage`, `TestSmoke_UnknownCommandExec`	unit (env/flag parsing) + smoke (builds+execs the real binary as a subprocess,

File	Representative functions	Type
		asserts real exit codes)
`cmd/ota-device-emu/main_test.go`	`TestSmoke_MissingHardwareID`, `TestSmoke_Help`, `TestSmoke_BadFlag`	smoke (subprocess exec, real exit codes/ stderr)
`cmd/ota-server/main_test.go`	`TestGetEnvDefault`, `TestSmoke_BadConfigExits`	unit + smoke
`tools/loadtest/main_test.go`	`TestRun_AgainstRealServer` (drives `run()` against a real `httptest` server, asserts p50/ p99 internally consistent), `TestSmoke_Selftest`, `TestRun_Non2xxCounted`, `TestPercentile`, `TestSummarize`	unit (percentile math) + smoke (build+exec) + light benchmarking-harness self-test — **NOT itself a DDoS/load test of the real `ota-server` binary**, only of a throwaway 1-line 200-OK handler

1.2 `internal/api/` — the bulk of the suite (46 files)

- **Integration-style (real `gin.Engine` + real `NewServer(...)` + real in-memory `store.Repository`, driven via `httptest.ResponseRecorder`, no mocks of the system under test per §11.4.27):** essentially every `handlers_*_test.go` file — `handlers_auth_test.go` (`TestLogin`, `TestRefreshRotation`, `TestRBACForbidsWrongRole`), `handlers_artifact_test.go`/`handlers_artifact_parts_test.go` (artifact upload + ed25519 signature verification), `handlers_device_test.go`, `handlers_release_test.go`, `handlers_deployment_test.go`, `handlers_rollout_test.go`, `handlers_delta_test.go`, `handlers_recall_test.go`, `handlers_telemetry_test.go`, `handlers_group_test.go`, `handlers_project_test.go`, `handlers_audit_test.go`, `handlers_branches_test.go`, `handlers_error_paths_test.go`, `handlers_widen_test.go`, `handlers_repo_error_test.go`, `handlers_client_test.go` / `handlers_client_antidowngrade_test.go`, `middleware_compression_test.go`, `coverage_*_test.go`, `embed_test.go`. This IS the project's "integration" layer for the API —

real router, real middleware chain (auth/RBAC/audit/compression), real business logic; only the persistence layer is the in-memory `Repository` (the actual MVP production implementation per architecture.md §4, not a mock).

- **Security-relevant, specifically citable:**
  - `internal/api/handlers_artifact_parts_test.go:83` `TestArtifactUploadIgnoresRequestSuppliedPubkey` — proves the exact trust boundary CLAUDE.md calls out (`handlers_artifact.go:resolvePublicKey`): an attacker-signed artifact with an attacker-supplied `pubkey` part is rejected (422), i.e. the request can never override the server-configured trusted key.
  - `internal/api/handlers_artifact_parts_test.go:110` `TestArtifactUploadNoTrustedKey`, `handlers_artifact_test.go:46` `TestArtifactUploadRejects`, `coverage_gap_test.go:342` `TestResolveSignatureAllBranches` — signature-verification edge cases.
  - `handlers_auth_test.go:96` `TestRBACForbidsWrongRole`, `handlers_group_test.go:99` `TestGroupRBAC`, `coverage_project_deployment_test.go` (8 functions covering `requireProjectAccess`: unauthenticated, admin-bypass, insufficient-role, viewer-can-view) — RBAC/authorization coverage.
  - `internal/api/embed_stress_chaos_test.go:376-439` (`dotdot_etc_passwd`, `dotdot_deep`, `assets_dotdot` cases inside `TestStressManagerSPA_BoundaryPaths`) — path-traversal-escape probe against the embedded manager-UI static file server, hard-asserts zero host-file leaks.
  - No security-header test; no cross-secret token-forgery test (see gaps §5). Note: the token scheme is fixed HMAC-SHA256 signed-opaque, not a general multi-algorithm JWT parser, so classic "alg=none" confusion does not structurally apply (positive finding, not a gap).
- **Stress + chaos, specifically citable:**
  - `chaos_test.go` (4 funcs): `TestChaosAuthBadPayload`, `TestChaosAuthHugePayload`, `TestChaosConcurrentMutation`, `TestChaosStoreRestart`.
  - `stress_test.go` (3 funcs): `TestStressConcurrentAuth`, `TestStressConcurrentRelease`, `TestStressConcurrentDevice`.
  - `resilience_test.go` (5 funcs): `TestStressConcurrentGroupCreate`, `TestStressSustainedReads`, `TestStressConcurrentMembershipNoLostUpdates`, **`TestDDoSFloodStaysUpAndRecovers`** (6000 requests / 64 workers against `/healthz` + a post-flood authenticated write, asserts 0

unexpected non-200/429 and immediate post-flood responsiveness — this is the file's own **explicit DDoS-class test**), `TestChaosRepoFaultDegradesAndRecovers` (injected repo fault → graceful 500 → clears → 200 recovery).
    - `embed_stress_chaos_test.go` (6 funcs, largest single stress+chaos file): `TestStressManagerSPA_SustainedMixedLoad` (480 concurrent requests across 4 response classes + **goroutine-leak check** via `runtime.NumGoroutine()` before/after, `leak-tolerance<=4`), `TestStressManagerSPA_ConcurrentContention`, `TestStressManagerSPA_BoundaryPaths` (incl. the path-traversal probe above), `TestChaosManagerSPA_OddMethodsAndHeaders`, `TestChaosManagerSPA_RawMalformedWire`, `TestChaosManagerSPA_ConcurrentConnPressure`.
    - `rate_limit_test.go` (2 funcs): `TestMaxInflightShedsUnderFlood` (cap=1, 300 concurrent, proves 429-shedding + post-flood recovery works **when the cap is enabled**), `TestMaxInflightDisabledByDefault` (cap=0 → never sheds — **documents that `HELIX_MAX_INFLIGHT` defaults to 0/disabled**, confirmed at `internal/config/config.go:126`).
    - `TestDDoSFloodStaysUpAndRecovers`'s own doc comment states the **honest finding**, verbatim: *"the MVP has NO rate-limiting — every request is served (no 429s) … Recommendation (tracked): add a rate-limit / concurrency-cap middleware before public exposure."* `rate_limit_test.go` is that follow-up: it proves the control **works when turned on**, but does not itself prove the shipped default deployment turns it on (see gap G-DDOS-1).
- **Benchmarking:** `bench_test.go` — `BenchmarkHealthz`, `BenchmarkGroupCreate`, `BenchmarkGroupList`, `BenchmarkClientUpdateNoDeployment` (4 of the repo's 7 total `Benchmark*` funcs).
- **Concurrency / race-deadlock specific:**
    - Deadlock-timeout guard pattern (`select { case <-done: ; case <-time.After(30*time.Second): t.Fatalf("… possible deadlock") }`) present in `embed_stress_chaos_test.go`'s `TestStressManagerSPA_SustainedMixedLoad`.
    - `handlers_parked_resilience_test.go` (5 funcs): `TestResilienceConcurrentGroupPagination`, `TestResilienceConcurrentTelemetryPagination`, `TestResilienceConcurrentSeedAndRead` — concurrent-correctness (no lost updates / no torn reads) under real goroutine fan-out.

1.3 `internal/device/` (applyport — A/B slot switch, ed25519 signing, client)

- `applyport_test.go` (28 funcs) + `applyport_mutation_test.go` (21 funcs, explicitly named "mutation" — §1.1-style paired assertions) + `coverage_test.go` / `coverage_extra_test.go`.
- Security-relevant: `TestSignatureVerifier_ValidSignature/InvalidSignature/TamperedPayload/NoKeyConfigured/BadSignatureHex/BadSignatureLength`, `TestMutationSignatureUsesRealEd25519`, `TestMutationSignatureTamperedPayloadCaught`, `TestRealEd25519Signature` — real ed25519 crypto exercised, not mocked.
- Concurrency: `TestMutationConcurrentAccessSafe`, `TestSlotDevice_ConcurrentAccess`.
- e2e-ish: `TestFullLifecycle_LoginRegisterCheck` (login → register → check-for-update against a real `httptest` server).

1.4 `internal/deviceemu/` (device emulator — the client half of the OTA protocol)

- `emulator_test.go`: `TestEmulatorFullLifecycle`, `TestEmulatorRunOnce`, `TestEmulatorReportFailure`, `TestEmulatorSelfServesDeploymentID`.
- **`recall_recovery_test.go:33 TestRecallRecoveryE2E`** — the strongest single **e2e** test in the repo: boots a real in-process `httptest` control plane, an operator stages release v1.1.0 + deployment, a real `deviceemu.Client` applies it, fails its post-apply health-check, the operator forward-fix-recalls to v1.2.0, the device re-checks and recovers — every step read back over real HTTP responses (§11.4.107-style liveness, not a single green line).
- `scaling_test.go`: `TestScalingConcurrentRegisterCheck`, `TestScalingConcurrentFullLifecycle` — concurrency at the device-fleet level.
- `error_paths_test.go` (29 funcs) — exhaustive negative-path coverage (bad JSON, transport errors, unexpected status codes) of the emulator's HTTP client.

1.5 `internal/fabric/`, `internal/health/`, `internal/config/` — small, pure-unit packages

- `config_test.go` (`TestLoadDefaults`, `TestLoadOverrides`, `TestLoadInvalidValues`), `health_test.go` (`TestLiveAlwaysTrue`, `TestReadyDefaultTrue`, `TestReadyProbe`) — genuine **unit** tests, no HTTP, no store.

- `fabric/registry_test.go` + `registry_branches_test.go` — lease-exclusivity concurrency (`TestRegistryLeaseExclusivity`) + evidence-store branch coverage.

1.6 `internal/rollout/` and `internal/store/` — the persistence + rollout-engine seam

- **Default-build (no tag), run in this audit:** `scenario_test.go`, `service_test.go`, `store_test.go` (rollout); `contract_test.go`, `memory_*_test.go`, `bench_test.go` (store) — all exercise the **in-memory** implementations (the real MVP-wired `store.Repository` per architecture.md §4, not a mock).
- **`//go:build integration`-gated (verified present, NOT executed in this audit — see §2):** `postgres_integration_test.go`, `postgres_coverage_integration_test.go`, `postgres_fault_integration_test.go`, `postgres_fabric_integration_test.go`, `faultproxy_test.go`, `pg_itest_lock_test.go` in both `internal/store/` and `internal/rollout/`. These boot a **real PostgreSQL** via the `digital.vasic.containers` submodule (per §11.4.74/§11.4.76 — never ad-hoc podman) and prove the pgx `Repository`/`PostgresStore` satisfy the identical behavioural contract as the in-memory implementation, including a real TCP fault-injection proxy (`faultproxy_test.go`) for chaos-class driver-fault coverage (`TestPostgresStoreDriverFaults_Integration`, `TestPostgresDriverFaults_Integration`).
- `TestFabricLeaseExclusivityGuard`, `TestReleaseFabricLeaseWithOtherActiveLease` — concurrency/exclusivity guarantees on the rollout fabric.

1.7 `internal/transport/` — HTTP/3 + HTTP/2 dual transport

- `transport_test.go:65` `TestDualTransportServesH3AndH2` — real dual-stack serve test.
- `transport_branches_test.go`, `transport_coverage_test.go` — construction/shutdown error-path branches (`TestNewWrapsHTTP3ConstructionError`, `TestShutdownReturnsHTTP2Error`).

1.8 `tests/chaos/` and `tests/stress/` (top-level `server/tests/`, distinct from `internal/api`'s own chaos/stress files)

- `tests/chaos/chaos_test.go` (5 funcs): `TestChaosMalformedPayloads`, `TestChaosHugePayload`, `TestChaosConcurrentCreateSameResource`, `TestChaosConcurrentConflictingMutations`, `TestChaosSustainedFaultRecovery`.
- `tests/stress/stress_test.go` (4 funcs): `TestStressSustainedGroupCreate`, `TestStressConcurrentAuth`, `TestStressConcurrentDeviceRegistration`, `TestStressBoundaryAuthPayloads`.
- Both packages explicitly document (source-comment) "Uses the real in-memory store (no mocks per §11.4.27)" and write captured-evidence logs to `qa-results/stress_chaos/`.

---

2. Captured suite-run output (real execution, this host, 2026-07-09/10)

All commands run from `server/` with `go1.26.4`.

```
$ go build ./...
(no output — clean build)

$ go test ./... -count=1
ok      .../server/cmd/applyport            2.293s
ok      .../server/cmd/ota-device-emu        2.383s
ok      .../server/cmd/ota-server            2.817s
ok      .../server/internal/api              7.004s
?       .../server/internal/api/manager-dist  [no test files]
ok      .../server/internal/config           0.003s
ok      .../server/internal/device           0.069s
ok      .../server/internal/deviceemu         0.498s
ok      .../server/internal/fabric           0.003s
ok      .../server/internal/health           0.002s
ok      .../server/internal/rollout          0.004s
ok      .../server/internal/store            0.004s
ok      .../server/internal/transport        0.112s
ok      .../server/tests/chaos               0.072s
ok      .../server/tests/stress              0.012s
ok      .../server/tools/loadtest            2.075s
```

Result: 100% package PASS, 0 failures, 0 skips, default (non-integration) build.

```
$ go vet ./...
(no output — clean)

$ gofmt -l .
(no output — zero formatting drift across the whole module)

$ go test ./... -race -count=1
ok  .../server/cmd/applyport            3.805s
ok  .../server/cmd/ota-device-emu        3.795s
ok  .../server/cmd/ota-server            4.883s
ok  .../server/internal/api             22.727s
ok  .../server/internal/config           1.011s
ok  .../server/internal/device           1.266s
ok  .../server/internal/deviceemu         4.072s
ok  .../server/internal/fabric           1.014s
ok  .../server/internal/health           1.012s
ok  .../server/internal/rollout          1.012s
ok  .../server/internal/store            1.013s
ok  .../server/internal/transport        1.264s
ok  .../server/tests/chaos               1.181s
ok  .../server/tests/stress              1.092s
ok  .../server/tools/loadtest            3.582s
```

Result: 100% package PASS under `-race`, 0 data-race reports across every package in the module — this is real, direct race-detector evidence, not an inference.

```
$ go test ./... -bench=. -benchtime=1x -run '^$' -count=1
BenchmarkHealthz-2                    1   78399 ns/op    9440 B/
op   67 allocs/op
BenchmarkGroupCreate-2                1   77397 ns/op   23104 B/
op  184 allocs/op
BenchmarkGroupList-2                  1   45369 ns/op   25528 B/
op  136 allocs/op
BenchmarkClientUpdateNoDeployment-2   1   18689 ns/op    9440 B/
op   58 allocs/op
BenchmarkMemoryCreateGroup-2          1    6710 ns/op    1560 B/
op    9 allocs/op
BenchmarkMemoryFindDelta-2            1     381.0 ns/op     0 B/
op    0 allocs/op
```

```
BenchmarkMemoryListAudit-2            1   43416 ns/op   92464 B/
op   10 allocs/op
```

Confirmed: exactly 7 real benchmark functions exist (4 in `internal/api/bench_test.go`, 3 in `internal/store/bench_test.go`); each reports real ns/op + B/op + allocs/op from a real run (not a stub).

```
$ go vet -tags integration ./internal/store/... ./internal/
rollout/...
(no output — the //go:build integration files type-check clean)
```

Honest boundary: this confirms the integration-tagged test source compiles correctly against the current code. It does **NOT** confirm the tests pass — no real PostgreSQL / containers-submodule boot was performed in this audit run (that would require booting the `digital.vasic.containers` stack, out of scope for a read-only audit). This matches the companion 2026-06-23 audit's finding G1 (store/rollout Postgres path was "UNCONFIRMED PASS" at the Go-test level as of that date too).

No `Fuzz*` function exists anywhere in the module (`grep -c '^func Fuzz'` = 0).

---

3. §11.4.169 coverage matrix

§11.4.169 enumerates a 12-item closed test-type set: unit / integration / e2e / full-automation / Challenges / HelixQA / DDoS / security / stress+chaos / concurrency / race-deadlock / memory / benchmarking.

#	Type	Status	Citation
1	**unit**	**COVERED**	`internal/config/config_test.go` (`TestLoadDefaults`/`TestLoadOverrides`/`TestLoadInvalidValues`), `internal/health/health_test.go`, `tools/loadtest/main_test.go` (`TestPercentile`, `TestSummarize`, `TestMs`), `internal/device/applyport_test.go` (`TestParseHelixSlot`, `TestSha256Hex`-class), `internal/api/coverage_extra_test.go` (`TestMethodVerb`, `TestSingular`, `TestTruncate`) — pure functions, no HTTP/store.
2	**integration**	**COVERED** (in-memory	In-memory: virtually all `internal/api/handlers_*_test.go` (real Gin router + real middleware + real `store.Repository`). Postgres:

#	Type	Status	Citation
		path) / **PARTIAL** (Postgres path)	`internal/store/postgres_integration_test.go` + 5 sibling files + `internal/rollout/postgres_integration_test.go` + 4 siblings exist, are real (boot real Postgres via the containers submodule, no fakes), type-check clean under `-tags integration`, but were **NOT executed in this audit** (no live Postgres booted) — status for that specific path is UNCONFIRMED-BY-THIS-AUDIT, not MISSING.
3	**e2e**	**COVERED**	`internal/deviceemu/recall_recovery_test.go:33` `TestRecallRecoveryE2E` (full stage→apply→fail→recall→recover lifecycle over real HTTP against an in-process control plane); `internal/device/applyport_test.go:735` `TestFullLifecycle_LoginRegisterCheck`; `internal/deviceemu/emulator_test.go` `TestEmulatorFullLifecycle`. All in-process (`httptest`), not a separately-deployed binary.
4	**full-automation**	**PARTIAL** (inside `server/`)	Every test above is autonomous (no human in the loop, §11.4.98-clean) — `TestMain` in `cmd/*` builds and execs real binaries with real exit-code assertions. What's absent **inside `server/`** is a black-box run of the compiled `ota-server` binary driven over the network by an external harness (that exists at the **project level**, outside `server/` — see §4).
5	**Challenges**	**MISSING inside `server/`**	No HelixQA/Challenges wiring exists under `server/` itself. (A real, wired Challenge bank targeting this exact server DOES exist at the project root — `tools/helixqa/banks/helix_ota.yaml` — see §4; it is out of the audited `server/` module boundary.)
6	**HelixQA**	**MISSING inside `server/`**	Same as above — HelixQA integration lives at the project root, not inside the Go module.
7	**DDoS**	**PARTIAL**	`internal/api/resilience_test.go:268` `TestDDoSFloodStaysUpAndRecovers` (6000 req / 64 workers, asserts stay-up + recovery) + `internal/api/rate_limit_test.go` (`TestMaxInflightShedsUnderFlood` proves the shedding control works when enabled). Gap: the

#	Type	Status	Citation
			flood test's own comment documents the shipped default has **no active rate limiter** (`HELIX_MAX_INFLIGHT` defaults to `0`, `internal/config/config.go:126`), and no test in `server/` proves the deployed default *configuration* (as opposed to the code path) enables it.
8	**security**	**PARTIAL**	RBAC (`handlers_auth_test.go:96 TestRBACForbidsWrongRole`, `coverage_project_deployment_test.go` 8 funcs), artifact-signature trust boundary (`handlers_artifact_parts_test.go:83 TestArtifactUploadIgnoresRequestSuppliedPubkey` — direct proof of the exact boundary CLAUDE.md names), ed25519 signature tamper/no-key/bad-length guards (`internal/device/applyport_test.go` + `applyport_mutation_test.go`), path-traversal-escape probe (`embed_stress_chaos_test.go` dotdot cases). Gap: no cross-secret token-forgery test (a validly-shaped token signed with a different secret than configured — see §5 gap 5), no security-response-header test (no `X-Content-Type-Options`/`X-Frame-Options`/HSTS anywhere in `internal/`, confirmed by direct grep — none set, none tested), no auth brute-force/lockout test, no SQL-injection-string probe against the pgx path (only in-memory path is exercised in `server/`'s own suite). The token scheme itself is fixed HMAC-SHA256 signed-opaque (not a multi-algorithm JWT parser), so classic alg-confusion does not structurally apply — a positive finding.
9	**stress+chaos**	**COVERED**	`internal/api/chaos_test.go`, `stress_test.go`, `resilience_test.go`, `embed_stress_chaos_test.go` (6 funcs, incl. goroutine-leak + deadlock-timeout guard), `tests/chaos/chaos_test.go`, `tests/stress/stress_test.go` — real concurrent load, fault injection (repo faults, malformed/huge payloads, raw malformed wire bytes), all against the real in-memory store. Postgres-backed chaos (`internal/store/faultproxy_test.go`, real TCP fault-injection proxy) exists but is `-tags integration`-gated and not executed in this audit.

#	Type	Status	Citation
10	**concurrency**	**COVERED**	`TestChaosConcurrentMutation`, `TestStressConcurrentMembershipNoLostUpdates`, `TestResilienceConcurrentGroupPagination`/`ConcurrentTelemetryPagination`/`ConcurrentSeedAndRead`, `TestScalingConcurrentRegisterCheck`/`ConcurrentFullLifecycle`, `TestMutationConcurrentAccessSafe`, `TestSlotDevice_ConcurrentAccess`, `TestFabricLeaseExclusivityGuard` — no-lost-update / no-torn-read assertions under real goroutine fan-out.
11	**race-deadlock**	**COVERED**	`go test ./... -race -count=1` — **0 data races across every package**, captured in §2 (real, direct evidence, this run). Deadlock: explicit 30s-timeout `select` guard in `TestStressManagerSPA_SustainedMixedLoad`; every stress/chaos test uses bounded `sync.WaitGroup` + timeouts rather than unbounded blocking.
12	**memory**	**PARTIAL**	Only one memory-adjacent signal exists: `embed_stress_chaos_test.go`'s goroutine-leak check (`runtime.NumGoroutine()` before/after 480-request burst, `leak-tolerance<=4`) in `TestStressManagerSPA_SustainedMixedLoad`. No heap/RSS growth assertion (`runtime.ReadMemStats`) anywhere, no memory-ceiling test tied to the constitution's §12.6 60% host-memory budget, no test asserting bounded memory under a large/sustained request volume for the core API handlers (groups/devices/releases/deployments/rollout) — only the embed static-file handler is goroutine-leak-checked.
—	**benchmarking**	**COVERED** (micro) / **PARTIAL** (no regression baseline)	7 real `Benchmark*` functions (§2) — `BenchmarkHealthz`, `BenchmarkGroupCreate`, `BenchmarkGroupList`, `BenchmarkClientUpdateNoDeployment`, `BenchmarkMemoryCreateGroup`, `BenchmarkMemoryFindDelta`, `BenchmarkMemoryListAudit`. Gap: none of these are compared against a stored baseline / regression threshold — a benchmark that silently regresses 10× would not fail any gate. `tools/loadtest`

#	Type	Status	Citation
			provides a real macro HTTP-load harness (RPS + p50/p90/p99), but its own test (`TestRun_AgainstRealServer`) only exercises it against a 1-line throwaway handler, not the real `ota-server` binary, and is not itself a benchmark gate.

Summary: 6 of 12 types fully COVERED (unit, e2e, stress+chaos, concurrency, race-deadlock, + integration/benchmarking effectively-covered-with-caveats), 5 PARTIAL (integration-Postgres, DDoS, security, memory, benchmarking-regression), 2 MISSING when scoped strictly to `server/` (Challenges, HelixQA) — though both exist and are wired at the project level (§4). Full-automation is functionally present inside `server/` (every test is autonomous) but the network-level black-box automation of the compiled binary lives outside the module.

---

4. Honest supplementary finding: project-level black-box coverage of the server (outside `server/`)

Per §11.4.6 (no-guessing) this audit must not report a type as MISSING if real, wired coverage exists — even if it lives outside the strict `server/` directory the task scoped this audit to. The following were found (read-only, not re-executed by this audit) directly targeting the `server/` code and are germane to interpreting the matrix above correctly:

- `tools/helixqa/banks/helix_ota.yaml` (rev 6, 1033 lines) — a real, wired **HelixQA Challenge bank** (closes types 5/6 at the project level). It dispatches `tests/e2e/challenge_operational.sh` and sibling scripts against a **live, real `ota-server` process over real HTTP** (curl+jq, no mocks); its own doc comment states “THIS BANK IS NOT METADATA-ONLY.”
- `tests/e2e/*.sh` (`challenge_operational.sh`, `challenge_filters_pagination.sh`, `recall_lifecycle.sh`, `rollout_halt_safety.sh`, `pipeline_signed.sh` / `pipeline_signed_live.sh`, `telemetry_schema_live.sh`) with committed evidence (`RUN_EVIDENCE.txt`: 39 passed/0 failed/1 skipped, PASS; `RECALL_EVIDENCE.txt`: 35/0/0 PASS; `ROLLOUT_HALT_EVIDENCE.txt`: 47/0/0 PASS;

`FILTERS_PAGINATION_EVIDENCE.txt`: 50/0/0 PASS; `PIPELINE_EVIDENCE.txt`: 32/0/0 PASS, dated 2026-06-19 through 2026-06-24) — real black-box **e2e** / **full-automation** proof against a compiled binary.

- `tests/security/*.sh` (`security_probes.sh`, `security_probes_extended.sh`, `security_probes_filters.sh`, `trust_boundary_live.sh`, `recall_telemetry_probes.sh`, `saturation_live.sh`) — closes several of the **security**/**DDoS** gaps flagged in §3 at the project level: unauthenticated-access, RBAC, resource-ownership, malformed/tampered/reused-JWT, **SQL-ish**/**path-traversal**/**NoSQL injection-string probes** (`security_probes.sh` probe class E), oversized/malformed-JSON, and the exact artifact-signature trust-boundary proven live against real PostgreSQL (`trust_boundary_live.sh`, evidence under `docs/qa/20260623-trust-boundary-live/`, cases positive-accept / bad-signature-reject / request-supplied-key-ignored, all captured PASS). `saturation_live.sh` (evidence `docs/qa/20260623-saturation-live/`) is the direct closure of the DDoS gap: it explicitly documents (verbatim in its header) that the live default stack ships with `HELIX_MAX_INFLIGHT` unset (cap OFF) and proves BOTH that the cap works when a second capped instance is launched AND that the default (uncapped) live stack survives a bounded flood without crashing.
  - **Honest negative finding:** the plain `tests/security/RUN_EVIDENCE.txt` (as opposed to the `_EXTENDED`/`_FILTERS`/`_RECALL_TELEMETRY` variants) shows its **last recorded run ABORTED** ("server not healthy" / "address already in use", 2026-06-24) — i.e. not every script in this project-level suite has a clean latest run; the extended/filters/recall-telemetry variants do show clean PASS (26/0/0, 39/0/0, 28/0/0 respectively, dated 2026-06-19/21).
- `tests/chaos/chaos_live.sh` + `tests/stress/http_load_live.sh` — real chaos (`docs/qa/20260623-chaos-live/`: real `podman stop`/`podman start` of the live Postgres container mid-run, asserts clean 5xx degrade + full recovery + no data corruption on a survivor row) and real load (`docs/qa/20260623-http-load-live/`: measured 11,889 req/s, p50 1.945ms / p90 3.418ms / p99 5.136ms against `/healthz`) — this is genuine macro-benchmarking + DDoS-adjacent + chaos evidence, dated 2026-06-23, that a strict `server/`-only view would otherwise miss entirely.
- `docs/qa/20260623-e2e-live-system/` — boots the real compiled `ota-server` binary against a real PostgreSQL (20-table schema confirmed in `postgres_evidence.txt`) via `tests/lib/boot_real_system.sh`, i.e. indirect but real evidence that the `-tags`

`integration` Go code path (pgx `Repository`) functions correctly end-to-end, even though this audit did not itself execute the Go-level integration tests (§2).

Net effect on the matrix: types 5 (Challenges) and 6 (HelixQA) are MISSING only if interpreted as "must live inside `server/`" — the constitution's intent (a real, evidence-backed Challenge/HelixQA bank exercising the real system) is in fact satisfied at the project root. Types 7 (DDoS) and 8 (security) are better characterized as COVERED-at-project-level / PARTIAL-inside-`server/` once this evidence is accounted for. This does not change the `server/`-scoped matrix in §3 (which is what was asked for) but materially changes what should be prioritized in a production-readiness plan: the real gaps are memory-growth assertion and benchmark-regression-baseline, not "build a Challenge bank from scratch."

---

5. Prioritized gap list (risk-descending, §11.4.132) with concrete test descriptions

1. **[HIGHEST] Memory / heap-growth assertion for the core API handlers.** Nothing in `server/` asserts bounded memory growth for the frequently-hit handlers (groups/devices/releases/deployments/rollout/telemetry) under sustained load — only the embed static-file handler has a goroutine-leak check. **Concrete test:** a `TestMemory_SustainedAPILoadNoGrowth`-class test that runs N≥1000 iterations of a representative mixed-endpoint workload (mirroring `TestStressManagerSPA_SustainedMixedLoad`'s pattern but against `/api/v1/groups`, `/api/v1/devices`, `/api/v1/releases`), calls `runtime.GC()` + `runtime.ReadMemStats` before/after, and asserts heap-in-use growth stays under an evidence-calibrated bound (§11.4.107(13) — calibrated on this project's own fixtures, never a hardcoded literature threshold) plus a goroutine-count check identical to the existing embed pattern. **Effort: small (0.5-1 day)** — the pattern already exists in `embed_stress_chaos_test.go` and just needs porting to the API-handler surface.

2. **[HIGH] Run the `-tags integration` Postgres suite for real and capture the result inside `server/`'s own CI-equivalent evidence trail.** Currently `go vet -tags integration` type-checks clean but the suite has never been executed by this audit (and per the 2026-06-23 companion audit, not executed there either at the Go-test level — only the project-level shell e2e-live scripts exercise the compiled

binary against real Postgres). **Concrete action:** boot the `digital.vasic.containers` Postgres stack (per §11.4.74/§11.4.76, never ad-hoc podman) and run `go test -tags integration ./internal/store/... ./internal/rollout/...`, capturing pass/fail + timing as committed evidence under `docs/qa/<run-id>/`. **Effort: small (a few hours)** — the harness (`digital.vasic.containers/pkg/boot`) is already imported and used by the test files; this is purely an execution/evidence-capture gap, not a missing-test gap.

3. **[HIGH] Benchmark regression baseline.** 7 real benchmarks exist but nothing compares a new run against a stored baseline — a 10× latency regression in `BenchmarkGroupCreate` would pass silently. **Concrete test/tooling:** a `benchstat`-based (or hand-rolled JSON-baseline) gate that runs `go test -bench=. -benchtime=<fixed-N>` against a committed `benchmarks/baseline.json`, fails on regression beyond a calibrated tolerance (e.g. ±20%), and updates the baseline only on deliberate operator-approved commits. **Effort: medium (1-2 days)** including wiring + calibrating tolerances against this project's own numbers per §11.4.107(13).

4. **[MEDIUM] Security-response-header coverage.** No `X-Content-Type-Options`, `X-Frame-Options`, or HSTS header is set anywhere in `internal/` (confirmed absent by direct grep), and consequently nothing tests for it. **Concrete test:** once (if) a security-header middleware is added, a `TestSecurityHeadersPresentOnEveryResponse` asserting the header set on a representative sample of routes (200, 400, 401, 403, 404, 500 paths) including error responses, not just happy-path 200s. **Effort: small** for the test once the middleware exists; the middleware itself is a separate (non-audit) implementation task.

5. **[MEDIUM] Cross-secret token-forgery test.** Correction after reading `internal/api/token.go`: this is **not** a general JWT library with an `alg` header (no algorithm-negotiation vector exists to confuse — it is a fixed HMAC-SHA256 signed-opaque `<payload>.<sig>` scheme), so classic "alg=none"/alg-confusion does not structurally apply here; that is a genuine positive finding, not a gap. The real gap: `coverage_gap_test.go:232` `TestTokenVerifyEdgeCases` covers a garbage-string signature (`"bad-signature"`), too-many-parts, and bad-base64-payload, but no test constructs a **validly-formed** token — correct JSON claims, correct base64url encoding, correct-length HMAC-SHA256 signature — signed with a **different secret** than the server's configured one (e.g. simulating a leaked-old-secret or cross-tenant forgery attempt) and asserts `Verify` rejects it. **Concrete test:**

`TestTokenVerifySignedWithWrongSecretRejected` — mint a token with `NewTokenSigner([]byte("attacker-secret"))`, verify it against the server's real signer, assert rejection. **Effort: trivial (under an hour)** — `internal/api/token.go`'s `Mint`/`Verify` already provide everything needed.

6. **[MEDIUM] SQL/NoSQL-injection-string probe against the pgx-backed path, inside `server/`.** The project-level `tests/security/security_probes.sh` probe class E already does this against a live (in-memory-by-default) server; nothing in `server/`'s own Go suite runs injection-shaped strings (`' OR '1'='1`, `{"$gt":""}`, `../../etc/passwd` in path/query/body params) against handlers that ultimately reach the pgx `Repository` (once `-tags integration` is exercised per gap 2). **Concrete test:** extend `internal/store/postgres_integration_test.go`'s scenario with a small `TestPostgresInjectionStringsNeverExecuteAsSQL`-class case feeding injection-shaped strings as e.g. group names / device hardware IDs and asserting they round-trip as literal data (pgx parameterized queries should already make this a formality, but there is currently no test proving it against the real driver). **Effort: small (a few hours)**, contingent on gap 2's harness being runnable.

7. **[LOW] Default-configuration DDoS-protection assertion.** `rate_limit_test.go` proves the cap works when explicitly enabled; nothing in `server/` (nor, per the honest finding in §4, the currently-shipped `system.compose.yml`) proves or requires `HELIX_MAX_INFLIGHT` to be non-zero by default. This is arguably a deployment-config decision rather than a test gap, but a `TestConfig_MaxInflightHasSafeProductionDefault`-class test (or a documented, deliberate `Won't-fix`/accepted-risk decision per §11.4.112 if unlimited concurrency is genuinely intended for the MVP) would close the ambiguity. **Effort: trivial (config decision + one assertion)**, but requires an operator decision on the intended default posture — flagged here, not silently resolved.

8. **[LOW] Native Go fuzzing (`func Fuzz...`).** Zero fuzz targets exist anywhere in the module. Candidate surfaces: the multipart artifact-upload metadata parser (`ArtifactUploadMetadata` JSON decode), the delta/version-string parsers, and the JWT bearer-token parser (`internal/api/coverage_gap_test.go`'s `TestBearerTokenEdgeCases` already hand-enumerates edge cases that a fuzz corpus would discover automatically and extend). **Effort: small per target (a few hours each)** — Go's built-in fuzzing (`go test -fuzz=Fuzz...`) requires no new dependency.

Sources verified

This audit is based entirely on (a) direct reading of `server/**/*_test.go` source files in this repository on 2026-07-09/10, and (b) real command execution (`go build`, `go test`, `go vet`, `gofmt`, `go test -bench`) on this host on the same dates, plus read-only inspection of pre-existing project artifacts (`docs/research/test-coverage-audit-20260623/REPORT.md`, `tools/helixqa/banks/helix_ota.yaml`, `tests/e2e/*.sh`, `tests/security/*.sh`, `tests/chaos/chaos_live.sh`, `tests/stress/http_load_live.sh`, and their `docs/qa/2026062*-*-live/` evidence directories). No external web sources were consulted — this is an internal repository audit, not a documentation-currency check, so §11.4.99 (latest-online-source cross-reference) does not apply. No file outside `docs/research/server_test_type_coverage_audit_20260709/` was modified. No git command was run.